

DATA STRUCTURES



Submitted to

Mr. Bilal Bukhari

Submitted by

Laraib Khalid

(fa25-bcs-059)

COMSATS University Islamabad,

Attock Campus.

SINGLY LINKED LISTS

Code:

```
#include <iostream>

using namespace std;

struct Node
{
    int data;
    Node* next;
};

// Insert node at end

void insert(Node*& head, int value)
{
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL)
    {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != NULL)
    {
        temp = temp->next;
```

```
}

temp->next = newNode;
}
// Reverse display using loop
void reverseUsingLoop(Node* head)
{
    int count = 0;
    Node* temp = head;
    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }
    int arr[count];
    temp = head;
    for (int i = 0; i < count; i++)
    {
        arr[i] = temp->data;
        temp = temp->next;
    }
    cout << "Reverse using Loop: ";
    for (int i = count - 1; i >= 0; i--)
    {
        cout << arr[i] << " ";
    }
}
```

```
        cout << endl;
    }
    // Reverse display using recursion
    void reverseUsingRecursion(Node* head)
    {
        if (head == NULL)
            return;

        reverseUsingRecursion(head->next);
        cout << head->data << " ";
    }
    int main()
    {
        Node* head = NULL;
        insert(head, 10);
        insert(head, 20);
        insert(head, 30);
        insert(head, 40);
        insert(head, 50);
        cout << "Original List: ";
        Node* temp = head;
        while (temp != NULL)
        {
            cout << temp->data << " ";
            temp = temp->next;
        }
    }
```

```
    cout << endl;

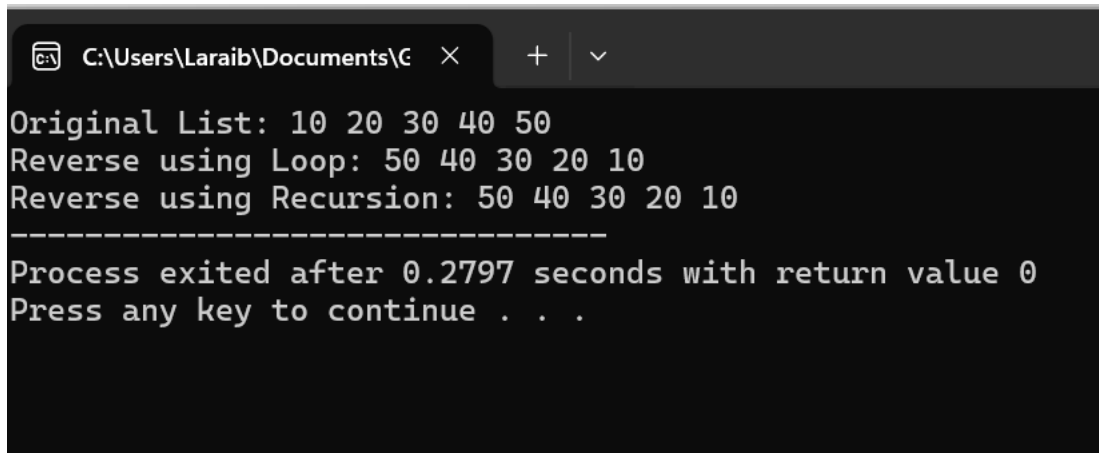
    reverseUsingLoop(head);

    cout << "Reverse using Recursion: ";

    reverseUsingRecursion(head);

    return 0;
}
```

RESULT



```
C:\Users\Laraib\Documents\C  ×  +  v
Original List: 10 20 30 40 50
Reverse using Loop: 50 40 30 20 10
Reverse using Recursion: 50 40 30 20 10
-----
Process exited after 0.2797 seconds with return value 0
Press any key to continue . . .
```

Code:

```
#include <iostream>

using namespace std;

struct Node
{
    int data;

    Node* next;
};
```

```
// Insert node at end

void insert(Node*& head, int value)
{
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL)
    {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}

// Display linked list

void display(Node* head)
{
    Node* temp = head;
    while (temp != NULL)
    {
        cout << temp->data << " ";
        temp = temp->next;
    }
}
```

```
    }  
    cout << endl;  
}  
  
// Merge two lists into a third new list  
Node* mergeLists(Node* head1, Node* head2)  
{  
    Node* newHead = NULL;  
    Node* tail = NULL;  
    // Copy first list  
    Node* temp = head1;  
    while (temp != NULL)  
    {  
        Node* newNode = new Node;  
        newNode->data = temp->data;  
        newNode->next = NULL;  
        if (newHead == NULL)  
        {  
            newHead = newNode;  
            tail = newNode;  
        }  
        else  
        {  
            tail->next = newNode;  
            tail = newNode;  
        }  
    }  
}
```

```
        temp = temp->next;
    }
    // Copy second list
    temp = head2;
    while (temp != NULL)
    {
        Node* newNode = new Node;
        newNode->data = temp->data;
        newNode->next = NULL;
        if (newHead == NULL)
        {
            newHead = newNode;
            tail = newNode;
        }
        else
        {
            tail->next = newNode;
            tail = newNode;
        }
        temp = temp->next;
    }
    return newHead;
}

int main()
{
    Node* head1 = NULL;
```

```
Node* head2 = NULL;

Node* head3 = NULL;

// First list

insert(head1, 10);

insert(head1, 20);

insert(head1, 30);

// Second list

insert(head2, 40);

insert(head2, 50);

insert(head2, 60);

cout << "First List: ";

display(head1);


cout << "Second List: ";

display(head2);

// Merge

head3 = mergeLists(head1, head2);

cout << "Third List after Merging: ";

display(head3);

return 0;

}
```

```
C:\Users\Laraib\Documents\G  X  +  v
First List: 10 20 30
Second List: 40 50 60
Third List after Merging: 10 20 30 40 50 60

-----
Process exited after 0.2448 seconds with return value 0
Press any key to continue . . . |
```

CODE

```
#include <iostream>

using namespace std;

struct Node
{
    int data;
    Node* next;
};

// Insert node at end
void insert(Node*& head, int value)
{
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;
```

```
if (head == NULL)
```

```
{
```

```
    head = newNode;
```

```
    return;
```

```
}
```

```
Node* temp = head;
```

```
while (temp->next != NULL)
```

```
{
```

```
    temp = temp->next;
```

```
}
```

```
temp->next = newNode;
```

```
}
```

```
// Display linked list
```

```
void display(Node* head)
```

```
{
```

```
    Node* temp = head;
```

```
while (temp != NULL)
```

```
{
```

```
    cout << temp->data << " ";
```

```
    temp = temp->next;
```

```
}
```

```
cout << endl;
```

```
}
```

```
// Find multiple occurrences
```

```
void findOccurrences(Node* head, int value)
```

```
{
```

```
    Node* temp = head;
```

```
    int position = 1;
```

```
    int count = 0;
```

```
    while (temp != NULL)
```

```
    {
```

```
        if (temp->data == value)
```

```
        {
```

```
            cout << "Value found at position: "
```

```
                << position << endl;
```

```
            count++;
```

```
        }
```

```
        temp = temp->next;
```

```
        position++;
```

```
    }
```

```
    if (count == 0)
    {
        cout << "Value not found." << endl;
    }
    else
    {
        cout << "Total occurrences: " << count << endl;
    }
}
```

```
int main()
{
    Node* head = NULL;
```

```
    // Creating list
```

```
    insert(head, 10);
```

```
    insert(head, 20);
```

```
    insert(head, 30);
```

```
    insert(head, 20);
```

```
    insert(head, 40);
```

```
    insert(head, 20);
```

```
    insert(head, 50);
```

```
    cout << "Linked List: ";
```

```
    display(head);
```

```
// Search value

int value = 20;

cout << "Searching for: " << value << endl;

findOccurrences(head, value);

return 0;
}
```

RESULT

```
Linked List: 10 20 30 20 40 20 50
Searching for: 20
Value found at position: 2
Value found at position: 4
Value found at position: 6
Total occurrences: 3

-----
Process exited after 0.2604 seconds with return value 0
Press any key to continue . . . |
```